

# Deep Learning Final Exam Study Guide

120 Comprehensive Practice Questions, Key Definitions & Mathematical Formulas

## USEFUL STUDY INFORMATION & DEFINITIONS: How to Use This Printable Study Guide

- Coverage: 12 core topics covering Dive into Deep Learning (D2L), video lectures, and lab notebooks.
- Structure: Useful Information & Formulas box -> 10 Practice Questions per topic (120 total) -> Verified Answer Key with Explanations.
- Google Docs Compatibility: Formatted with standard headings, margins, callout tables, and Unicode math for seamless Google Docs upload and printing.

## Topic 1: Linear Neural Networks & Regression (D2L Ch 3.1, 3.4)

### USEFUL STUDY INFORMATION & DEFINITIONS: Linear Regression & Vectorization Mechanics

**Linear Model Equation:**  $\hat{y} = w_1 x_1 + w_2 x_2 + \dots + w_d x_d + b = \mathbf{w}^T \mathbf{x} + b$

**Vectorized Batch Matrix Notation:**  $\hat{\mathbf{Y}} = \mathbf{X} \mathbf{w} + \mathbf{b}$  ( $\mathbf{X}$  in  $\mathbb{R}^{(n \times d)}$ ,  $\mathbf{w}$  in  $\mathbb{R}^{(d \times 1)}$ ,  $\mathbf{b}$  in  $\mathbb{R}^{(1 \times 1)}$ )

**Mean Square Error (MSE) Loss:**  $L(\mathbf{w}, \mathbf{b}) = (1 / 2n) * \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)})^2$

**Minibatch Gradient Descent Update:**  $\mathbf{w} \leftarrow \mathbf{w} - (\eta / |\mathbf{B}|) * \sum_{i \in \mathbf{B}} \text{grad}_{\mathbf{w}} l^{(i)}(\mathbf{w}, \mathbf{b})$

#### Key Concepts:

- Vectorization: Replaces Python loops with GPU/CPU matrix multiplication, scaling throughput by 100x+.
- MLE Equivalence: Minimizing MSE is mathematically equivalent to Maximum Likelihood Estimation under zero-mean Gaussian noise.
- Closed-Form Solution:  $\mathbf{w}^* = (\mathbf{X}^T \mathbf{X})^{-1} * \mathbf{X}^T * \mathbf{Y}$  (Normal Equation).

**Question 1:** What does the Mean Square Error (MSE) measure in a regression model?

- A.** The sum of absolute differences between predicted and actual values.
- B.** The probability of a prediction being correct.
- C.** The ratio of correct predictions to total predictions.
- D.** The average squared difference between predicted and actual values.

**Answer: D. The average squared difference between predicted and actual values.** — MSE computes the average squared residual, heavily penalizing large deviations.

**Question 2:** Why is vectorization essential when implementing linear regression in PyTorch or NumPy?

- A.** It eliminates the need for computing gradients.
- B.** It replaces slow Python loops with highly parallelized linear algebra BLAS/CUDA routines.

- C. It automatically prevents the model from overfitting.
- D. It guarantees the loss function becomes non-convex.

**Answer: B. It replaces slow Python loops with highly parallelized linear algebra BLAS/CUDA routines.** — Vectorized matrix multiplication leverages SIMD hardware acceleration for fast minibatch processing.

**Question 3:** Under what statistical assumption is minimizing Mean Squared Error equivalent to Maximum Likelihood Estimation (MLE)?

- A. Assuming observation noise is normally (Gaussian) distributed with zero mean.
- B. Assuming target labels follow a Bernoulli distribution.
- C. Assuming features are non-linear and independent.
- D. Assuming data has zero variance.

**Answer: A. Assuming observation noise is normally (Gaussian) distributed with zero mean.** — Maximizing Gaussian log-likelihood reduces directly to minimizing the sum of squared errors.

**Question 4:** What is the closed-form analytical solution for linear regression parameters  $w^*$  given design matrix  $X$  and target vector  $Y$ ?

- A.  $w^* = X^T * Y$
- B.  $w^* = (X^T * X)^{-1} * X^T * Y$
- C.  $w^* = (X * X^T)^{-1} * Y$
- D.  $w^* = \text{softmax}(X * Y)$

**Answer: B.  $w^* = (X^T * X)^{-1} * X^T * Y$**  — Setting the gradient of squared loss to zero yields the normal equation  $w^* = (X^T * X)^{-1} * X^T * Y$ .

**Question 5:** What is the primary role of the gradient descent algorithm in training neural networks?

- A. To initialize weights using random normal distributions.
- B. To optimize model parameters by updating them in the direction of steepest loss decrease.
- C. To evaluate model generalization accuracy on validation sets.
- D. To dynamically change input feature dimensions.

**Answer: B. To optimize model parameters by updating them in the direction of steepest loss decrease.** — Gradient descent moves parameters opposite the loss gradient vector to reach local/global loss minima.

**Question 6:** In minibatch stochastic gradient descent, what does the hyperparameter minibatch size ( $|B|$ ) control?

- A. The total number of parameters in the model.
- B. The number of training samples processed in a single forward/backward pass to compute a gradient estimate.
- C. The depth of the neural network architecture.
- D. The magnitude of weight decay applied.

**Answer: B. The number of training samples processed in a single forward/backward pass to compute a gradient estimate.** — Minibatch size balances stochastic gradient variance and GPU parallel computational throughput.

**Question 7:** If a linear regression model has 784 input features and 1 output, what are the dimensions of the weight matrix  $W$  and bias  $b$  for a batch of 256 examples?

- A.  $W$  in  $\mathbb{R}^{(784 \times 256)}$ ,  $b$  in  $\mathbb{R}^{(256 \times 1)}$
- B.  $W$  in  $\mathbb{R}^{(784 \times 1)}$ ,  $b$  in  $\mathbb{R}^{(1)}$
- C.  $W$  in  $\mathbb{R}^{(256 \times 784)}$ ,  $b$  in  $\mathbb{R}^{(784)}$
- D.  $W$  in  $\mathbb{R}^{(1 \times 784)}$ ,  $b$  in  $\mathbb{R}^{(256)}$

**Answer: B.  $W$  in  $\mathbb{R}^{(784 \times 1)}$ ,  $b$  in  $\mathbb{R}^{(1)}$**  — Weights map  $d=784$  features to 1 output, giving shape  $(784, 1)$  and scalar bias  $b$ .

**Question 8:** What happens if the learning rate ( $\eta$ ) in gradient descent is set too large?

- A. The model converges instantly to the global minimum.
- B. The gradient step overshoots the minimum, causing loss oscillation or divergence to infinity/NaN.
- C. Training becomes slower but more accurate.
- D. The weights automatically decay to zero.

**Answer: B. The gradient step overshoots the minimum, causing loss oscillation or divergence to infinity/NaN.** — Large learning rates cause updates to jump across loss valleys, creating numerical divergence.

**Question 9:** Why do we reshuffle (randomly permute) the training dataset at the start of every epoch during minibatch SGD?

- A. To increase the total number of parameters.
- B. To prevent biased gradient updates caused by sample ordering patterns in the data.
- C. To force the learning rate to decay automatically.
- D. To transform regression into classification.

**Answer: B. To prevent biased gradient updates caused by sample ordering patterns in the data.** — Shuffling guarantees stochastic independence of minibatches across epochs.

**Question 10:** In PyTorch, what is the computational advantage of using `nn.MSELoss()` over a manual Python loop calculating squared differences?

- A. `nn.MSELoss()` runs in C++/CUDA and integrates directly with autograd for efficient derivative calculation.
- B. `nn.MSELoss()` automatically tunes the learning rate.
- C. `nn.MSELoss()` eliminates backpropagation.
- D. `nn.MSELoss()` requires no target labels.

**Answer: A. `nn.MSELoss()` runs in C++/CUDA and integrates directly with autograd for efficient derivative calculation.** — Built-in PyTorch loss modules execute optimized low-level CUDA kernels and build automatic differentiation graph nodes.

## Topic 2: Multilayer Perceptrons & Activation Functions (D2L Ch 5.1, 5.3)

## USEFUL STUDY INFORMATION & DEFINITIONS: MLP Architecture & Non-Linear Activation Functions

### MLP Formulation:

Hidden Layer:  $H = \sigma(X * W^{(1)} + b^{(1)})$  ( $X$  in  $\mathbb{R}^{(n \times d)}$ ,  $W^{(1)}$  in  $\mathbb{R}^{(d \times h)}$ )

Output Layer:  $O = H * W^{(2)} + b^{(2)}$  ( $W^{(2)}$  in  $\mathbb{R}^{(h \times q)}$ )

### Activation Functions:

- ReLU:  $f(x) = \max(0, x)$ ,  $f'(x) = 1$  ( $x > 0$ ) else 0. Solves vanishing gradient for  $x > 0$ .
- Sigmoid:  $\sigma(x) = 1 / (1 + \exp(-x))$ ,  $\sigma'(x) = \sigma(x)(1 - \sigma(x))$ . Saturates for large  $|x|$ .
- Tanh:  $\tanh(x) = (\exp(x) - \exp(-x)) / (\exp(x) + \exp(-x))$ , zero-centered in  $(-1, 1)$ .

### Key Concepts:

- Non-linearity: Stacking linear layers without activation collapses into a single linear model.
- Universal Approximation: 1 hidden layer with non-linear activation can approximate any continuous function.

**Question 11:** What happens if a neural network with multiple hidden layers uses strictly linear activation functions ( $f(x) = x$ ) across all layers?

- A. The network becomes exponentially more expressive.
- B. The entire multi-layer network mathematically collapses into a single linear model.
- C. Gradients explode immediately.
- D. It solves non-linearly separable problems like XOR.

**Answer: B. The entire multi-layer network mathematically collapses into a single linear model.** — Composition of affine transformations  $W_2(W_1 X + b_1) + b_2$  is strictly affine  $W_{\text{net}} X + b_{\text{net}}$ .

**Question 12:** Select ALL critical characteristics of the ReLU activation function.

- A. It outputs zero for negative input values.
- B. It is non-differentiable at exactly zero.
- C. It has a constant derivative of 1 for positive inputs, mitigating vanishing gradients.
- D. All of the above.

**Answer: D. All of the above.** — ReLU is  $\max(0, x)$ , zero for  $x < 0$ , non-differentiable subgradient corner at  $x=0$ , and derivative 1 for  $x > 0$ .

**Question 13:** What is a major limitation of the Sigmoid activation function in deep networks?

- A. It causes gradient saturation (vanishing gradients) when inputs are far from zero.
- B. It cannot be used for binary classification output layers.
- C. Its output is unbounded, causing activations to explode.
- D. It is non-differentiable everywhere.

**Answer: A. It causes gradient saturation (vanishing gradients) when inputs are far from zero.** — As  $|x|$  grows large, sigmoid derivative  $\sigma'(x) = \sigma(x)(1 - \sigma(x))$  approaches 0, killing gradient flow.

**Question 14:** What is the primary advantage of using backpropagation in training deep neural networks?

- A. It eliminates the need for training data.
- B. It allows for efficient, systematic computation of gradients using the calculus chain rule.
- C. It guarantees that the network will never overfit.
- D. It reduces the parameter count of the network.

**Answer: B. It allows for efficient, systematic computation of gradients using the calculus chain rule.** — Backpropagation reuses intermediate activation derivatives layer-by-layer backwards, eliminating redundant work.

**Question 15:** In a 1-hidden-layer MLP classifying 10 categories from 784 input features with 256 hidden units, what is the shape of the weight matrix  $W^{(1)}$ ?

- A.  $784 \times 10$
- B.  $784 \times 256$
- C.  $256 \times 10$
- D.  $10 \times 256$

**Answer: B.  $784 \times 256$**  —  $W^{(1)}$  connects 784 input features to 256 hidden neurons.

**Question 16:** What is the phenomenon of 'dying ReLU' neurons?

- A. Neurons whose weights become infinite due to high learning rates.
- B. Neurons that output zero for all dataset examples, receiving zero gradients and permanently stopping learning.
- C. Neurons that oscillate between positive and negative values.
- D. Neurons that automatically convert into linear units.

**Answer: B. Neurons that output zero for all dataset examples, receiving zero gradients and permanently stopping learning.** — If a large gradient step pushes a neuron's pre-activation below zero for all inputs, its gradient becomes 0 and it dies.

**Question 17:** How does the Tanh activation function differ from the Sigmoid activation function?

- A. Tanh outputs values in  $(-1, 1)$  and is zero-centered, whereas Sigmoid outputs values in  $(0, 1)$ .
- B. Tanh is strictly linear for positive values.
- C. Tanh cannot be differentiated.
- D. Sigmoid is zero-centered while Tanh is not.

**Answer: A. Tanh outputs values in  $(-1, 1)$  and is zero-centered, whereas Sigmoid outputs values in  $(0, 1)$ .** — Zero-centered Tanh outputs mitigate directional gradient bias during backpropagation.

**Question 18:** According to the Universal Approximation Theorem, what is required for a 1-hidden-layer neural network to approximate any continuous function?

- A. At least 100 hidden layers with linear activations.
- B. A non-linear activation function and a sufficient number of hidden units.
- C. Convolutional layers with max pooling.
- D. Batch normalization before every layer.

**Answer: B. A non-linear activation function and a sufficient number of hidden units.** —

Universal approximation holds for single-hidden-layer MLPs provided activation is non-linear and width is sufficient.

**Question 19:** What is forward propagation in a neural network?

- A. Calculating partial derivatives backward from the loss function.
- B. Sequentially calculating and storing intermediate activations from input layer to output layer.
- C. Updating weight parameters using gradient descent.
- D. Randomly setting 50% of weights to zero.

**Answer: B. Sequentially calculating and storing intermediate activations from input layer to output layer.** — Forward pass computes pre-activations and activations layer by layer to obtain final predictions.

**Question 20:** In a computational graph for backpropagation, what does a node represent?

- A. An operation or variable, with directed edges representing data flow dependencies.
- B. The learning rate schedule.
- C. A GPU device index.
- D. The cross-entropy loss formula only.

**Answer: A. An operation or variable, with directed edges representing data flow dependencies.** — Computational graphs explicitly express variable dependency chains required for autograd.

### Topic 3: Generalization, Regularization & Dropout (D2L Ch 3.7, 5.5, 5.6)

#### USEFUL STUDY INFORMATION & DEFINITIONS: Generalization, Weight Decay & Dropout Regularization

Generalization Gap = Training Error - Validation Error

##### Overfitting vs Underfitting:

- Overfitting: Low training error, high validation error (model memorizes noise).
- Underfitting: High training error, high validation error (model lacks capacity).

##### L2 Regularization (Weight Decay):

Loss:  $L_{\text{reg}}(w, b) = L(w, b) + (\lambda / 2) * ||w||_2^2$

Update:  $w \leftarrow (1 - \eta * \lambda) * w - \eta * \text{grad}_w L(w, b)$  ( $(1 - \eta * \lambda) < 1$  shrinks weights)

##### Dropout Regularization:

Inverted Dropout Training:  $\hat{h} = 0$  with prob  $p$ , else  $h / (1 - p)$ . Keeps  $E[\hat{h}] = h$  during training.

Test Mode: Disable dropout ( $\hat{h} = h$ ), ensuring deterministic prediction.

**Question 21:** In the weight decay update rule  $w \leftarrow (1 - \eta * \lambda) * w - \eta * \text{grad } L$ , what does the factor  $(1 - \eta * \lambda)$  accomplish?

- A. It increases all weight values proportionally.
- B. It shrinks the weight vector toward zero at every step before subtracting the gradient step.

- C. It normalizes the learning rate automatically.
- D. It sets negative weights to zero.

**Answer: B. It shrinks the weight vector toward zero at every step before subtracting the gradient step.** — Weight decay scales down parameters continuously, constraining overall weight norm.

**Question 22:** What is the primary mechanism by which Dropout prevents overfitting in neural networks?

- A. It permanently removes layers from the network architecture.
- B. Randomly deactivating neurons forces the network to learn robust, distributed representations rather than relying on co-adapted features.
- C. It decreases the learning rate when validation loss rises.
- D. It scales input features to zero mean.

**Answer: B. Randomly deactivating neurons forces the network to learn robust, distributed representations rather than relying on co-adapted features.** — Dropout breaks co-adaptation by preventing hidden units from depending on fixed subnetworks.

**Question 23:** Why is 'Inverted Dropout' (scaling activations by  $1 / (1 - p)$  during training) preferred in modern frameworks like PyTorch?

- A. It allows dropout to be kept active during test time.
- B. It ensures expected value of activations remains unchanged, requiring no modification during test/inference time.
- C. It increases model training speed by 2x.
- D. It converts non-linear layers into linear layers.

**Answer: B. It ensures expected value of activations remains unchanged, requiring no modification during test/inference time.** — Inverted scaling preserves expected activation energy during training, making test time a identity pass.

**Question 24:** What symptom indicates that a model is severely overfitting?

- A. Both training loss and validation loss remain high and constant.
- B. Training loss drops close to zero while validation loss increases, creating a wide generalization gap.
- C. Validation loss is lower than training loss.
- D. Training accuracy is lower than test accuracy.

**Answer: B. Training loss drops close to zero while validation loss increases, creating a wide generalization gap.** — Divergence between low training error and high validation error is the textbook definition of overfitting.

**Question 25:** How should Dropout be applied during the test (inference) phase?

- A. Deactivate 50% of neurons randomly.
- B. Set dropout probability  $p = 1.0$ .
- C. Disable dropout completely so all neurons contribute deterministically to the output.
- D. Apply dropout only to the output layer.

**Answer: C. Disable dropout completely so all neurons contribute deterministically to the output.** — During inference, all neurons are kept active to produce exact deterministic outputs.

**Question 26:** Why are bias parameters (b) typically EXCLUDED from L2 weight decay regularization?

- A. Bias parameters do not cause overfitting because they do not control feature sensitivity/variance.
- B. Biases are non-differentiable.
- C. Regularizing biases causes gradients to explode.
- D. PyTorch does not support bias updating.

**Answer: A. Bias parameters do not cause overfitting because they do not control feature sensitivity/variance.** — Biases control linear offsets rather than feature curvature/sensitivity.

**Question 27:** Which strategy is effective for combating underfitting?

- A. Increasing model capacity (adding layers or units) or reducing regularization strength.
- B. Increasing dropout probability  $p$  to 0.9.
- C. Applying stronger L2 weight decay.
- D. Stopping training at epoch 1.

**Answer: A. Increasing model capacity (adding layers or units) or reducing regularization strength.** — Underfitting indicates insufficient capacity; adding units or loosening regularization allows better fit.

**Question 28:** What does Early Stopping accomplish during neural network training?

- A. It stops training as soon as validation loss stops improving to prevent overfitting.
- B. It stops training when training loss reaches exactly zero.
- C. It reduces hidden layer width dynamically.
- D. It removes negative weights.

**Answer: A. It stops training as soon as validation loss stops improving to prevent overfitting.** — Early stopping halts training when generalization error reaches its minimum on the validation set.

**Question 29:** What happens if the L2 regularization hyperparameter ( $\lambda$ ) is set to an extremely large value?

- A. The model achieves 100% accuracy.
- B. The weights are heavily suppressed toward zero, causing severe underfitting.
- C. The model transforms into a decision tree.
- D. Training speed becomes infinite.

**Answer: B. The weights are heavily suppressed toward zero, causing severe underfitting.** — Over-regularizing forces parameters to zero, stripping the network of expressive power.

**Question 30:** Where in a Multilayer Perceptron is Dropout typically applied?

- A. Directly on raw input features only.
- B. On the outputs of hidden layers, after activation functions.
- C. On final softmax probabilities.
- D. On target class labels.

**Answer: B. On the outputs of hidden layers, after activation functions.** — Dropout is placed after non-linear hidden activations to regularize internal representations.

## Topic 4: Optimization Algorithms: SGD, Momentum & Adam (D2L Ch 12.1, 12.3–12.6, 12.10)

### USEFUL STUDY INFORMATION & DEFINITIONS: Optimization: SGD, Momentum, Adam & Learning Rate Scaling

#### Optimization Formulas:

- SGD:  $w_t = w_{t-1} - \eta * g_t$  ( $g_t$  = minibatch gradient)
- Momentum:  $v_t = \beta * v_{t-1} + g_t$ ,  $w_t = w_{t-1} - \eta * v_t$  (dampens oscillations in narrow valleys)
- Adam (Adaptive Moment Estimation):
  - First Moment (Mean):  $m_t = \beta_1 * m_{t-1} + (1 - \beta_1) * g_t$
  - Second Moment (Variance):  $v_t = \beta_2 * v_{t-1} + (1 - \beta_2) * g_t^2$
  - Bias Corrections:  $\hat{m}_t = m_t / (1 - \beta_1^t)$ ,  $\hat{v}_t = v_t / (1 - \beta_2^t)$
  - Update:  $w_t = w_{t-1} - \eta * \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$

#### Key Concepts:

- Linear Scaling Rule: When increasing batch size by factor  $k$ , scale learning rate  $\eta$  by  $k$ .
- Saddle Points & Ravines: Non-convex loss landscapes contain saddle points where gradients vanish; momentum builds velocity to escape.

**Question 31:** What is the key innovation of the Adam optimizer compared to standard SGD with Momentum?

- A.** It eliminates the need for backpropagation.
- B.** It maintains exponential moving averages of both gradients and squared gradients, adapting learning rate per parameter.
- C.** It guarantees finding the global minimum in non-convex losses.
- D.** It uses second-order Hessian matrix inversion.

**Answer: B. It maintains exponential moving averages of both gradients and squared gradients, adapting learning rate per parameter.** — Adam combines Momentum (first moment  $m_t$ ) and RMSProp (second moment  $v_t$ ) to compute adaptive per-parameter step sizes.

**Question 32:** Why are bias correction steps  $\hat{m}_t = m_t / (1 - \beta_1^t)$  and  $\hat{v}_t = v_t / (1 - \beta_2^t)$  required in Adam?

- A.** To prevent learning rates from becoming negative.
- B.** To correct for zero-initialization bias in early training iterations when  $t$  is small.
- C.** To convert regression into classification.
- D.** To scale parameters by weight decay.

**Answer: B. To correct for zero-initialization bias in early training iterations when  $t$  is small.** — Since  $m_0=0$  and  $v_0=0$ , uncorrected moment estimates are biased toward zero at early steps  $t$ .

**Question 33:** How does Momentum optimization help accelerate training in narrow loss valleys (ravines)?

- A. It increases the batch size automatically.
- B. It accumulates past velocity, dampening high-frequency oscillations across steep walls while accelerating progress along the valley floor.
- C. It sets all negative gradients to zero.
- D. It eliminates the need for learning rates.

**Answer: B. It accumulates past velocity, dampening high-frequency oscillations across steep walls while accelerating progress along the valley floor.** — Opposing transverse oscillations cancel out in velocity  $v_t$  while persistent directional gradients build speed.

**Question 34:** According to the Linear Scaling Rule, how should the learning rate ( $\eta$ ) be adjusted when the minibatch size ( $|B|$ ) is multiplied by  $k$ ?

- A. The learning rate should remain unchanged.
- B. The learning rate should be decreased by  $1/k$ .
- C. The learning rate should be increased proportionally by  $k$ .
- D. The learning rate should be squared.

**Answer: C. The learning rate should be increased proportionally by  $k$ .** — Larger minibatches produce lower gradient variance, allowing larger stable gradient steps  $\eta_{\text{new}} = k * \eta$ .

**Question 35:** What standard default values are commonly used for Adam hyperparameter coefficients ( $\beta_1$ ,  $\beta_2$ ,  $\epsilon$ )?

- A.  $\beta_1 = 0.5$ ,  $\beta_2 = 0.5$ ,  $\epsilon = 1.0$
- B.  $\beta_1 = 0.9$ ,  $\beta_2 = 0.999$ ,  $\epsilon = 1e-8$
- C.  $\beta_1 = 0.0$ ,  $\beta_2 = 0.0$ ,  $\epsilon = 0.1$
- D.  $\beta_1 = 0.99$ ,  $\beta_2 = 0.9$ ,  $\epsilon = 1e-3$

**Answer: B.  $\beta_1 = 0.9$ ,  $\beta_2 = 0.999$ ,  $\epsilon = 1e-8$**  — Standard Adam defaults set first moment momentum  $\beta_1=0.9$ , second moment  $\beta_2=0.999$ , and smoothing  $\epsilon=1e-8$ .

**Question 36:** What is a saddle point in a non-convex loss landscape?

- A. A point where loss is zero.
- B. A point where zero gradient occurs, but function curves upward in some directions and downward in others.
- C. The absolute global minimum.
- D. A point where activations saturate.

**Answer: B. A point where zero gradient occurs, but function curves upward in some directions and downward in others.** — Saddle points have zero gradient without being local minima; momentum helps escape them.

**Question 37:** In PyTorch, what function call updates optimizer parameters using computed gradients?

- A. `optimizer.zero_grad()`
- B. `loss.backward()`

- C. `optimizer.step()`
- D. `model.forward()`

**Answer: C. `optimizer.step()`** — `optimizer.step()` executes parameter updates  $w \leftarrow w - \text{update\_rule}$  using stored `.grad` tensors.

**Question 38:** Why is `optimizer.zero_grad()` called before `loss.backward()` in PyTorch training loops?

- A. To reset weight parameters to zero.
- B. To clear accumulated gradients from previous iterations before calculating new gradients.
- C. To disable backpropagation.
- D. To scale the learning rate.

**Answer: B. To clear accumulated gradients from previous iterations before calculating new gradients.** — PyTorch accumulates `.grad` tensors by default; failing to zero gradients causes incorrect accumulated updates.

**Question 39:** What is a Learning Rate Warmup schedule?

- A. Starting with a small learning rate for initial epochs and gradually increasing it to initial eta before decaying.
- B. Heating the GPU before running training loops.
- C. Setting learning rate to infinity for epoch 1.
- D. Disabling momentum for the first 100 steps.

**Answer: A. Starting with a small learning rate for initial epochs and gradually increasing it to initial eta before decaying.** — Warmup prevents early training instability when random weight initializations produce large noisy gradients.

**Question 40:** What is the primary difference between Convex and Non-Convex optimization?

- A. Convex optimization has a single global minimum with no local minima traps, whereas non-convex losses have multiple local minima and saddle points.
- B. Non-convex optimization is solvable in one step.
- C. Convex problems cannot be solved with gradient descent.
- D. Non-convex losses have no gradients.

**Answer: A. Convex optimization has a single global minimum with no local minima traps, whereas non-convex losses have multiple local minima and saddle points.** — Deep learning loss surfaces are non-convex, presenting local minima, ravines, and saddle points.

## Topic 5: Convolutional Neural Networks Foundations (D2L Ch 7.1–7.6)

### USEFUL STUDY INFORMATION & DEFINITIONS: CNN Foundations: Convolutions, Pooling, Padding & Stride

#### Convolutional Layer Output Dimensions Formula:

Output Height:  $H_{\text{out}} = \text{floor}((H_{\text{in}} - K_h + 2 * P_h) / S_h) + 1$

Output Width:  $W_{\text{out}} = \text{floor}((W_{\text{in}} - K_w + 2 * P_w) / S_w) + 1$

where  $K$  = Kernel Size,  $P$  = Padding,  $S$  = Stride.

**Receptive Field:** The spatial region of raw input pixels that influences a specific downstream neuron's output.

- Stacking two 3x3 conv layers gives an effective receptive field of 5x5.

**Key Properties:**

- Cross-Correlation vs Convolution: CNNs compute 2D cross-correlation (sliding dot products) without kernel flipping.
- Translation Invariance: Shift in input features yields corresponding shift in output feature maps.
- Max Pooling: Takes maximum value in pooling window, providing local translation invariance and spatial downsampling.

**Question 41:** Given an input image of size 32x32, kernel size 5x5, padding 2, and stride 1, what is the output spatial size ( $H_{out} \times W_{out}$ )?

- A. 28 x 28
- B. 32 x 32
- C. 30 x 30
- D. 16 x 16

**Answer: B. 32 x 32** —  $H_{out} = \text{floor}((32 - 5 + 2*2)/1) + 1 = \text{floor}(31/1) + 1 = 32$ . Padding  $P=(K-1)/2$  preserves spatial size.

**Question 42:** What are two fundamental visual inductive biases incorporated into Convolutional Neural Networks?

- A. Recursion and self-attention.
- B. Locality (pixels close together are highly correlated) and Translation Invariance (features are relevant anywhere in image).
- C. Fully connected all-to-all connectivity and random masking.
- D. Global pooling and linear decay.

**Answer: B. Locality (pixels close together are highly correlated) and Translation Invariance (features are relevant anywhere in image).** — CNNs enforce weight sharing (translation invariance) and local receptive fields (locality).

**Question 43:** What does 2x2 Max Pooling with stride 2 accomplish on a 64x64 feature map?

- A. Increases spatial size to 128x128.
- B. Divides feature map into non-overlapping 2x2 windows, taking maximum activation and downsampling spatial dimensions to 32x32.
- C. Converts all activations to zero.
- D. Adds 2 extra channels.

**Answer: B. Divides feature map into non-overlapping 2x2 windows, taking maximum activation and downsampling spatial dimensions to 32x32.** — 2x2 pooling with stride 2 halves height and width (64 -> 32) while extracting dominant local features.

**Question 44:** What is the receptive field of a neuron in a convolutional neural network?

- A. The number of output channels in a layer.

- B. The specific local region of raw input pixels that influences the activation of that neuron.
- C. The learning rate of the conv filter.
- D. The size of the GPU memory buffer.

**Answer: B. The specific local region of raw input pixels that influences the activation of that neuron.** — Receptive field measures the spatial extent of input context visible to a specific feature map neuron.

**Question 45:** What is the receptive field of a single neuron after stacking TWO consecutive 3x3 convolutional layers (stride 1, no padding)?

- A. 3 x 3
- B. 5 x 5
- C. 6 x 6
- D. 9 x 9

**Answer: B. 5 x 5** — Layer 1 3x3 kernel sees 3x3 input. Layer 2 3x3 kernel over Layer 1 spans  $3 + (3-1) = 5 \times 5$  input pixels.

**Question 46:** How many trainable parameter weights (excluding biases) are in a Conv2D layer with 32 input channels, 64 output channels, and a 3x3 kernel?

- A.  $32 \times 64 \times 3 \times 3 = 18,432$
- B.  $32 + 64 + 3 + 3 = 102$
- C.  $64 \times 3 \times 3 = 576$
- D.  $32 \times 3 \times 3 = 288$

**Answer: A.  $32 \times 64 \times 3 \times 3 = 18,432$**  — Parameter count =  $(C_{in} * C_{out} * K_h * K_w) = 32 * 64 * 3 * 3 = 18,432$  weights.

**Question 47:** Why are fully connected (dense) layers impractical for direct high-resolution image processing compared to conv layers?

- A. Fully connected layers cannot calculate non-linear activations.
- B. Fully connected layers connect every input pixel to every neuron, causing parameter count explosion and extreme overfitting.
- C. Fully connected layers are slower than matrix inverse.
- D. Dense layers do not support backpropagation.

**Answer: B. Fully connected layers connect every input pixel to every neuron, causing parameter count explosion and extreme overfitting.** — A 1000x1000 RGB image (3M inputs) to 1000 hidden units requires 3 billion weights in a dense layer, whereas 3x3 conv kernels require only ~100 parameters.

**Question 48:** What does 1x1 Convolution accomplish in modern CNN architectures?

- A. It downsamples spatial height and width.
- B. It performs cross-channel feature pooling/combination and parameter reduction without altering spatial dimensions.
- C. It replaces max pooling entirely.
- D. It acts as Gaussian smoothing.

**Answer: B. It performs cross-channel feature pooling/combination and parameter reduction without altering spatial dimensions.** — 1x1 conv computes linear combinations across channels at each pixel position, adjusting channel depth  $C_{in} \rightarrow C_{out}$ .

**Question 49:** What is the spatial effect of applying 'Valid' padding (padding  $P = 0$ ) vs 'Same' padding in Conv2D?

- A. Valid padding keeps spatial dimensions constant; Same padding reduces dimensions.
- B. Valid padding adds no padding (reducing spatial dimensions by  $K-1$ ); Same padding pads inputs so output spatial size equals input size (when stride=1).
- C. Valid padding doubles image resolution.
- D. Both paddings yield identical outputs.

**Answer: B. Valid padding adds no padding (reducing spatial dimensions by  $K-1$ ); Same padding pads inputs so output spatial size equals input size (when stride=1).** — Same padding uses  $P = (K-1)/2$  to maintain spatial size  $H_{out} = H_{in}$  for stride 1.

**Question 50:** How does Average Pooling differ from Max Pooling?

- A. Average Pooling computes the mean value of pixels in the window rather than selecting the maximum activation.
- B. Average Pooling increases spatial dimensions.
- C. Average Pooling includes learnable weights.
- D. Max Pooling is non-differentiable everywhere.

**Answer: A. Average Pooling computes the mean value of pixels in the window rather than selecting the maximum activation.** — Average pooling smooths activations across windows, whereas max pooling selects dominant peak features.

## Topic 6: Modern CNN Architectures & Batch Normalization (D2L Ch 8.1, 8.2, 8.5, 8.6)

### USEFUL STUDY INFORMATION & DEFINITIONS: Modern CNN Architectures: AlexNet, VGG, Batch Normalization & ResNet

#### Batch Normalization Formula (Mini-batch $B$ ):

1. Mean:  $\mu_B = (1 / |B|) * \sum_{i \in B} x^{(i)}$
2. Variance:  $\sigma_B^2 = (1 / |B|) * \sum_{i \in B} (x^{(i)} - \mu_B)^2$
3. Normalize:  $\hat{x}^{(i)} = (x^{(i)} - \mu_B) / \sqrt{\sigma_B^2 + \epsilon}$
4. Scale & Shift:  $y^{(i)} = \gamma * \hat{x}^{(i)} + \beta$  ( $\gamma, \beta$  are learnable parameters)

#### ResNet Residual Learning:

- Residual Block:  $y = F(x) + x$  (shortcut / skip connection carries identity  $x$  directly)
- Degradation Problem: In plain deep networks, adding layers causes training/test error to INCREASE due to gradient degradation. ResNet residual mapping  $F(x) = y - x$  makes identity mapping trivial ( $F(x) \rightarrow 0$ ), enabling 100+ layer networks.

#### Modern CNN Breakthroughs:

- AlexNet (2012): ReLU, Dropout, GPUs, 11x11 convs beat traditional CV on ImageNet.

- VGG (2014): Stacked small 3x3 conv filters with max pooling to build deep uniform blocks.

**Question 51:** In ResNet, what fundamental mapping does the residual block formula  $y = F(x) + x$  represent?

- A. Multiplying input  $x$  by activation function  $F$ .
- B. Adding input shortcut  $x$  to learned residual mapping  $F(x)$ , allowing unimpeded gradient flow through identity path.
- C. Subtracting input from output.
- D. Converting 2D images to 1D sequences.

**Answer: B. Adding input shortcut  $x$  to learned residual mapping  $F(x)$ , allowing unimpeded gradient flow through identity path.** — Identity skip connection allows gradients to flow directly back through addition nodes without vanishing.

**Question 52:** What was the 'Degradation Problem' that motivated the invention of Residual Networks (ResNet)?

- A. Networks became too fast on GPUs.
- B. Adding more layers to a plain deep network caused BOTH training error and test error to INCREASE due to optimization difficulty.
- C. Convolutional filters exploded to infinity.
- D. Image resolutions degraded.

**Answer: B. Adding more layers to a plain deep network caused BOTH training error and test error to INCREASE due to optimization difficulty.** — Plain 56-layer networks performed worse on training data than 20-layer networks; ResNet solved this via identity mappings.

**Question 53:** In Batch Normalization formula  $y = \gamma \cdot \hat{x} + \beta$ , what is the role of learnable parameters  $\gamma$  and  $\beta$ ?

- A. They replace activation functions.
- B. They allow the network to learn any optimal mean ( $\beta$ ) and scale ( $\gamma$ ), restoring original representational capacity if needed.
- C. They control minibatch size.
- D. They adjust learning rate per epoch.

**Answer: B. They allow the network to learn any optimal mean ( $\beta$ ) and scale ( $\gamma$ ), restoring original representational capacity if needed.** —  $\gamma$  and  $\beta$  allow the network to undo zero-mean unit-variance normalization if an unnormalized distribution is optimal.

**Question 54:** How does Batch Normalization behave DIFFERENTLY during training vs testing (inference)?

- A. During training, it uses minibatch mean/variance; during testing, it uses moving average (running mean/variance) computed during training.
- B. It is disabled completely during training.
- C. It randomly drops 50% of activations during testing.
- D. It uses batch size = 1 during training.

**Answer: A. During training, it uses minibatch mean/variance; during testing, it uses moving average (running mean/variance) computed during training.** — Test-time BatchNorm uses fixed exponential moving averages (`running_mean`, `running_var`) for deterministic evaluation.

**Question 55:** What are key benefits of incorporating Batch Normalization in deep CNNs?

- A. Stabilizes training, allows higher learning rates, reduces sensitivity to weight initialization, and provides mild regularization.
- B. Eliminates the need for loss functions.
- C. Reduces dataset size by 50%.
- D. Converts conv layers to linear layers.

**Answer: A. Stabilizes training, allows higher learning rates, reduces sensitivity to weight initialization, and provides mild regularization.** — BatchNorm smooths loss landscapes, enabling fast convergence with higher learning rates.

**Question 56:** What design principle characterized the VGG architecture (Simonyan & Zisserman, 2014)?

- A. Using large 11x11 conv filters with stride 4.
- B. Stacking uniform blocks of small 3x3 conv filters (stride 1, padding 1) with 2x2 max pooling.
- C. Eliminating all non-linear activation functions.
- D. Using only fully connected layers.

**Answer: B. Stacking uniform blocks of small 3x3 conv filters (stride 1, padding 1) with 2x2 max pooling.** — VGG proved that stacking deep 3x3 conv blocks achieves higher non-linearity and fewer parameters than large 7x7/11x11 filters.

**Question 57:** What major historical innovations enabled AlexNet (Krizhevsky et al., 2012) to win ImageNet?

- A. ReLU activations, Dropout regularization, GPU acceleration, and data augmentation.
- B. Self-attention and Transformers.
- C. Batch Normalization and Residual Connections.
- D. Linear regression and analytical inverse.

**Answer: A. ReLU activations, Dropout regularization, GPU acceleration, and data augmentation.** — AlexNet combined GPUs, ReLU, Dropout, and heavy augmentation to outperform hand-crafted CV pipelines.

**Question 58:** In a 1x1 conv projection in ResNet when spatial dimensions or channel counts change between residual blocks, what is the shortcut path?

- A. Identity mapping  $x$ .
- B. A 1x1 convolution with stride 2 to match output channel count and spatial dimensions:  $y = F(x) + W_s * x$ .
- C. Dropout with  $p=0.9$ .
- D. Max pooling with padding 5.

**Answer: B. A 1x1 convolution with stride 2 to match output channel count and spatial dimensions:  $y = F(x) + W_s * x$ .** — When spatial size or channel count changes, a 1x1 conv projection aligns shortcut dimensions  $W_s \times$  with  $F(x)$ .

**Question 59:** Where is Batch Normalization typically placed in a standard Convolutional Block?

- A. After Dense output layer only.
- B. Between Conv2D operation and ReLU activation: Conv2D -> BatchNorm -> ReLU.
- C. After Softmax probabilities.
- D. Before raw image input.

**Answer: B. Between Conv2D operation and ReLU activation: Conv2D -> BatchNorm -> ReLU.**

— Standard placement normalizes linear pre-activations  $Z = W * X + b$  before passing through non-linear ReLU.

**Question 60:** Why does Batch Normalization exhibit a mild regularization effect during training?

- A. Because it sets 50% of weights to zero.
- B. Because mini-batch mean and variance statistics introduce small random noise into activations during each training step.
- C. Because it multiplies learning rate by 0.1.
- D. Because it truncates parameter precision.

**Answer: B. Because mini-batch mean and variance statistics introduce small random noise into activations during each training step.** — Stochastic variation in sample mini-batch stats acts as additive noise, regularizing hidden features.

## Topic 7: Computer Vision: Image Augmentation & Fine-Tuning (D2L Ch 14.1, 14.2)

### USEFUL STUDY INFORMATION & DEFINITIONS: Computer Vision: Data Augmentation & Transfer Learning Fine-Tuning

#### Data Augmentation Techniques:

- Random Cropping & Resizing, Horizontal Flipping, Color Jittering (brightness, contrast, saturation), Random Rotation.
- Objective: Synthetically expands training set size, injecting realistic domain transformations to reduce overfitting.

#### Transfer Learning & Fine-Tuning Steps:

1. Select Source Pre-trained Model (e.g., ResNet-50 trained on 1.2M ImageNet images).
2. Replace Final Output Head: Swap 1000-class ImageNet fully connected layer for a new K-class target output head.
3. Initialize Weights: Initialize feature extractor backbone with pre-trained weights; initialize new output head randomly.
4. Training Strategy:
  - Small target dataset / high similarity: Freeze backbone weights ( $lr = 0$ ), train only output head.
  - Large target dataset / low similarity: Fine-tune all layers using small learning rate (e.g.,  $\eta_{backbone} = 0.1 * \eta_{head}$ ).

**Question 61:** When is Transfer Learning / Fine-Tuning MOST effective?

- A. When your target dataset is very small, but closely resembles the domain of the pre-trained model (e.g., ImageNet).
- B. When you have 50 million labeled target images.
- C. When target images are completely synthetic noise.
- D. When you want to train without any GPU.

**Answer: A. When your target dataset is very small, but closely resembles the domain of the pre-trained model (e.g., ImageNet).** — Transfer learning leverages low-level visual features (edges, textures) learned on massive datasets when target data is limited.

**Question 62:** What is the primary purpose of Image Data Augmentation during CNN training?

- A. To decrease the number of channels in the input image.
- B. To artificially inflate dataset diversity by applying random realistic transformations, reducing model overfitting.
- C. To speed up backpropagation by 10x.
- D. To convert RGB images to grayscale.

**Answer: B. To artificially inflate dataset diversity by applying random realistic transformations, reducing model overfitting.** — Augmentation introduces spatial/color invariance, preventing networks from memorizing exact pixel layouts.

**Question 63:** When fine-tuning a pre-trained ResNet on a new 5-class target task, how is the architecture modified?

- A. All convolutional layers are deleted.
- B. The original 1000-class fully connected output layer is replaced with a new randomly initialized 5-class linear layer.
- C. The input image size is fixed to 1x1.
- D. Batch normalization layers are removed.

**Answer: B. The original 1000-class fully connected output layer is replaced with a new randomly initialized 5-class linear layer.** — The feature extraction backbone is kept while the classification head is adapted to the target category count.

**Question 64:** Why should data augmentation transformations (e.g., random horizontal flips) be applied ONLY during training, NOT during test/validation evaluation?

- A. Because test evaluation requires deterministic, unaugmented inputs to reliably measure true model performance.
- B. Because GPUs cannot run augmentation during testing.
- C. Because test images have no channels.
- D. Because data augmentation reduces test accuracy to zero.

**Answer: A. Because test evaluation requires deterministic, unaugmented inputs to reliably measure true model performance.** — Validation/test metrics must evaluate exact, unmodified data samples without random perturbation.

**Question 65:** What fine-tuning learning rate strategy is recommended for pre-trained backbone parameters vs the new output head parameters?

- A. Use a much smaller learning rate (e.g., 0.1x) for pre-trained backbone weights and a larger learning rate for the new output head.
- B. Use a learning rate of 100.0 for backbone weights.
- C. Use zero learning rate for all layers permanently.
- D. Randomly pick learning rates every step.

**Answer: A. Use a much smaller learning rate (e.g., 0.1x) for pre-trained backbone weights and a larger learning rate for the new output head.** — A smaller learning rate preserves rich pre-trained backbone features while allowing the new head to learn rapidly.

**Question 66:** What does 'Freezing' a layer during fine-tuning mean?

- A. Setting layer activation values to absolute zero.
- B. Setting `requires_grad = False` for that layer, disabling gradient updates so its weights remain fixed during training.
- C. Deleting the layer from memory.
- D. Cooling the GPU temperature.

**Answer: B. Setting `requires_grad = False` for that layer, disabling gradient updates so its weights remain fixed during training.** — Freezing prevents backpropagation updates from altering pre-trained weight parameters.

**Question 67:** Which image augmentation technique would be INAPPROPRIATE for a digit recognition task (e.g., distinguishing 6 vs 9)?

- A. Random horizontal/vertical flipping or 180-degree rotation, because it alters the underlying semantic class label.
- B. Small random color jittering.
- C. Random cropping.
- D. Normalization.

**Answer: A. Random horizontal/vertical flipping or 180-degree rotation, because it alters the underlying semantic class label.** — Flipping/rotating a '6' transforms it into a '9', corrupting label semantics.

**Question 68:** Why do early layers in a deep pre-trained CNN generalize well across diverse computer vision tasks?

- A. Because early layers extract generic low-level visual primitives like Gabor-like edges, blobs, and textures.
- B. Because early layers contain 99% of all network parameters.
- C. Because early layers are fully connected.
- D. Because early layers do not perform convolutions.

**Answer: A. Because early layers extract generic low-level visual primitives like Gabor-like edges, blobs, and textures.** — Low-level edge/texture detectors are task-agnostic features common to almost all visual domains.

**Question 69:** What is Test-Time Augmentation (TTA)?

- A. Training a model with 1 million augmented images.

- B.** Generating multiple augmented versions of a single test image at inference time, running predictions on each, and averaging outputs.
- C.** Augmenting test label text.
- D.** Deleting test set samples.

**Answer: B. Generating multiple augmented versions of a single test image at inference time, running predictions on each, and averaging outputs.** — Ensembling predictions across subtle test augmentations lowers variance and boosts classification confidence.

**Question 70:** In PyTorch torchvision.transforms, why is transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]) standard practice?

- A.** It matches the exact channel mean and standard deviation normalization used when pre-training ImageNet models.
- B.** It converts RGB images to binary 0/1.
- C.** It resizes images to 224x224.
- D.** It replaces missing pixels with zeros.

**Answer: A. It matches the exact channel mean and standard deviation normalization used when pre-training ImageNet models.** — Matching pre-training channel normalization ensures input activations align with pre-trained weight distributions.

## Topic 8: Recurrent Neural Networks & BPTT (D2L Ch 9.4, 9.7)

### USEFUL STUDY INFORMATION & DEFINITIONS: Recurrent Neural Networks, Hidden States & BPTT

#### RNN State Update & Output Equations (Time step $t$ ):

Hidden State:  $h_t = \tanh(X_t * W_{xh} + h_{t-1} * W_{hh} + b_h)$  ( $h_t$  in  $\mathbb{R}^{(n \times h)}$ )

Output Vector:  $O_t = h_t * W_{hq} + b_q$  ( $O_t$  in  $\mathbb{R}^{(n \times q)}$ )

#### Backpropagation Through Time (BPTT):

- Unrolls sequence across  $T$  timesteps into a deep computational graph.
- Gradient Chain Rule across time:  $dh_t / dh_k = \prod_{j=k+1}^t (W_{hh}^T * \text{diag}(1 - \tanh^2(z_j)))$
- Vanishing & Exploding Gradients: Repeated matrix multiplication by  $W_{hh}$  causes gradients to vanish ( $||W_{hh}|| < 1$ ) or explode ( $||W_{hh}|| > 1$ ) over long sequences.
- Gradient Clipping: if  $||g|| > \theta$  then  $g \leftarrow (\theta / ||g||) * g$  (prevents exploding gradient updates).

**Question 71:** What is the primary defining feature of a Recurrent Neural Network (RNN) compared to a Feedforward MLP?

- A.** RNNs process all sequence tokens simultaneously in parallel.
- B.** RNNs maintain an internal hidden state  $h_t$  that persists and updates sequentially at each timestep, acting as temporal memory.
- C.** RNNs contain no weight parameters.
- D.** RNNs can only be used for image data.

**Answer: B. RNNs maintain an internal hidden state  $h_t$  that persists and updates sequentially at each timestep, acting as temporal memory.** — Recurrent hidden connections  $h_t = \tanh(X_t W_{xh} + h_{t-1} W_{hh} + b_h)$  pass temporal context forward.

**Question 72:** What is Backpropagation Through Time (BPTT)?

- A. An algorithm that reverses chronological order of dataset text.
- B. The gradient computation algorithm for RNNs created by unrolling the recurrent network across sequence timesteps into a deep graph.
- C. A method to train models without computing derivatives.
- D. A hardware timer on GPUs.

**Answer: B. The gradient computation algorithm for RNNs created by unrolling the recurrent network across sequence timesteps into a deep graph.** — BPTT applies the calculus chain rule backward through time across unrolled recurrent sequence steps.

**Question 73:** What causes the Vanishing Gradient problem when training standard vanilla RNNs on long sequences?

- A. Repeated matrix multiplication by recurrent weight matrix  $W_{hh}$  over  $T$  steps causes gradient components to decay exponentially toward zero when eigenvalues  $< 1$ .
- B. Learning rates automatically decaying to zero.
- C. Using batch size = 1.
- D. Data augmentation.

**Answer: A. Repeated matrix multiplication by recurrent weight matrix  $W_{hh}$  over  $T$  steps causes gradient components to decay exponentially toward zero when eigenvalues  $< 1$ .** — Gradient product  $\text{prod } W_{hh}^T$  shrinks exponentially over long time horizons  $T$ , preventing early weight updates.

**Question 74:** How does Gradient Clipping prevent training instability in RNNs?

- A. By setting all negative weights to zero.
- B. If gradient norm  $\|g\|$  exceeds threshold  $\theta$ , it rescales  $g \leftarrow (\theta / \|g\|) * g$ , capping step magnitude to prevent exploding gradients.
- C. By clipping input sequence length to 5 tokens.
- D. By dropping 50% of sequence words.

**Answer: B. If gradient norm  $\|g\|$  exceeds threshold  $\theta$ , it rescales  $g \leftarrow (\theta / \|g\|) * g$ , capping step magnitude to prevent exploding gradients.** — Gradient clipping constrains the maximum step vector magnitude without altering gradient direction.

**Question 75:** If an RNN hidden layer has 128 hidden units and input feature dimension is 64, what are the shapes of weight matrices  $W_{xh}$  and  $W_{hh}$ ?

- A.  $W_{xh}$  in  $\mathbb{R}^{(64 \times 128)}$ ,  $W_{hh}$  in  $\mathbb{R}^{(128 \times 128)}$
- B.  $W_{xh}$  in  $\mathbb{R}^{(128 \times 64)}$ ,  $W_{hh}$  in  $\mathbb{R}^{(64 \times 64)}$
- C.  $W_{xh}$  in  $\mathbb{R}^{(64 \times 64)}$ ,  $W_{hh}$  in  $\mathbb{R}^{(128 \times 128)}$
- D.  $W_{xh}$  in  $\mathbb{R}^{(128 \times 128)}$ ,  $W_{hh}$  in  $\mathbb{R}^{(128 \times 64)}$

**Answer: A.  $W_{xh}$  in  $\mathbb{R}^{(64 \times 128)}$ ,  $W_{hh}$  in  $\mathbb{R}^{(128 \times 128)}$**  —  $W_{xh}$  maps input 64 to 128 hidden units;  $W_{hh}$  connects previous hidden state 128 to 128 hidden units.

**Question 76:** What is Perplexity in language modeling?

- A. The percentage of misspelled words in text.
- B. The exponential of average cross-entropy loss  $\exp(L)$ , representing effective branching factor of next-token uncertainty (lower is better).
- C. The total number of parameters in an RNN.
- D. The time taken per training epoch.

**Answer: B. The exponential of average cross-entropy loss  $\exp(L)$ , representing effective branching factor of next-token uncertainty (lower is better).** — Perplexity =  $\exp(-1/N * \sum \log P(x_t))$  measures language model uncertainty; PPL=1 is perfect prediction.

**Question 77:** Why is sequential dependency in standard vanilla RNNs a computational bottleneck on modern GPUs?

- A. Because computing hidden state  $h_t$  strictly requires  $h_{t-1}$ , preventing parallel calculation across sequence timesteps during forward pass.
- B. Because RNNs use too much GPU memory.
- C. Because GPUs cannot perform matrix multiplication.
- D. Because text tokens cannot be vectorized.

**Answer: A. Because computing hidden state  $h_t$  strictly requires  $h_{t-1}$ , preventing parallel calculation across sequence timesteps during forward pass.** — Temporal dependency  $h_t = f(h_{t-1}, x_t)$  forces  $O(T)$  sequential processing steps, limiting GPU parallelism.

**Question 78:** In character-level language modeling with an RNN, how are input characters represented before being fed into the network?

- A. As raw ASCII integer values directly.
- B. As One-Hot encoded vectors or learnable dense embedding vectors mapped from vocabulary indices.
- C. As floating-point image matrices.
- D. As random normal numbers.

**Answer: B. As One-Hot encoded vectors or learnable dense embedding vectors mapped from vocabulary indices.** — Tokens are converted to vocabulary indices and projected via one-hot or embedding layers into continuous vectors.

**Question 79:** What happens if Truncated BPTT is used instead of Full BPTT on a sequence of 10,000 tokens?

- A. Gradients are computed across all 10,000 steps at once.
- B. Sequence is broken into shorter sub-sequences (e.g.,  $\tau = 35$  steps), limiting gradient backpropagation to  $\tau$  steps to save memory and stabilize training.
- C. The network forgets all weights.
- D. Training becomes non-deterministic.

**Answer: B. Sequence is broken into shorter sub-sequences (e.g.,  $\tau = 35$  steps), limiting gradient backpropagation to  $\tau$  steps to save memory and stabilize training.** — Truncated BPTT caps gradient graph unrolling to fixed window  $\tau$ , mitigating memory costs and vanishing gradients.

**Question 80:** Why does a vanilla RNN struggle to capture long-range temporal dependencies (e.g., context 100 steps prior)?

- A. Because hidden states overwrite memory at every step and vanishing gradients eliminate long-range error signals.
- B. Because vanilla RNNs contain no activation functions.
- C. Because cross-entropy loss cannot evaluate long sequences.
- D. Because GPUs discard context after 5 steps.

**Answer: A. Because hidden states overwrite memory at every step and vanishing gradients eliminate long-range error signals.** — Continuous non-linear updates without additive memory channels cause information decay over extended sequence steps.

## Topic 9: Modern RNNs: LSTMs & Gated Units (D2L Ch 10.1)

### USEFUL STUDY INFORMATION & DEFINITIONS: Modern RNNs: Long Short-Term Memory (LSTM) Architecture

#### LSTM Gated Memory Cell Mechanics (Time step $t$ ):

1. Forget Gate:  $F_t = \sigma(X_t * W_{xf} + H_{t-1} * W_{hf} + b_f)$  in  $(0, 1)$
2. Input Gate:  $I_t = \sigma(X_t * W_{xi} + H_{t-1} * W_{hi} + b_i)$  in  $(0, 1)$
3. Candidate Memory Cell:  $\tilde{C}_t = \tanh(X_t * W_{xc} + H_{t-1} * W_{hc} + b_c)$  in  $(-1, 1)$
4. Cell State Update:  $C_t = F_t (*) C_{t-1} + I_t (*) \tilde{C}_t$  (Additive conveyor belt;  $(*)$  = elementwise product)
5. Output Gate:  $O_t = \sigma(X_t * W_{xo} + H_{t-1} * W_{ho} + b_o)$  in  $(0, 1)$
6. Hidden State Output:  $H_t = O_t (*) \tanh(C_t)$

#### Key Distinction:

- Cell State  $C_t$ : Dedicated long-term internal memory belt running linearly through sequence with minimal modification.
- Hidden State  $H_t$ : Filtered short-term working memory output passed to downstream layers and next timestep.

**Question 81:** What is the main architectural innovation of an LSTM cell compared to a standard vanilla RNN?

- A. Replacing hidden states with convolutional layers.
- B. Introducing an internal Cell State ( $C_t$ ) and gating mechanisms (Forget, Input, Output gates) that explicitly regulate information flow.
- C. Eliminating all matrix multiplications.
- D. Processing sequences in parallel.

**Answer: B. Introducing an internal Cell State ( $C_t$ ) and gating mechanisms (Forget, Input, Output gates) that explicitly regulate information flow.** — LSTMs add a linear cell state conveyor belt  $C_t$  controlled by sigmoid gates to prevent vanishing gradients.

**Question 82:** In an LSTM cell, what is the specific mathematical function of the Forget Gate ( $F_t$ )?

- A. To compute cross-entropy loss.
- B. To output values in (0, 1) via sigmoid that multiply previous cell state  $C_{t-1}$  element-wise, deciding what information to erase.
- C. To randomly zero out weights during training.
- D. To scale the learning rate.

**Answer: B. To output values in (0, 1) via sigmoid that multiply previous cell state  $C_{t-1}$  element-wise, deciding what information to erase.** —  $F_t = \text{sigma}(\dots)$  outputs 0 (completely forget) to 1 (completely retain) multiplied by  $C_{t-1}$ .

**Question 83:** How does the Cell State ( $C_t$ ) update formula  $C_t = F_t (*) C_{t-1} + I_t (*) C_{\text{tilde}_t}$  mitigate vanishing gradients in LSTMs?

- A. By multiplying cell state by 100 at every step.
- B. The additive term (+) allows error gradients to flow back across time linearly with minimal decay when  $F_t$  approx 1.
- C. By replacing backpropagation with linear regression.
- D. By removing bias vectors.

**Answer: B. The additive term (+) allows error gradients to flow back across time linearly with minimal decay when  $F_t$  approx 1.** — Additive cell updates create an uninhibited gradient highway  $dC_t / dC_{t-1}$  approx  $F_t$ , preserving long-range error signals.

**Question 84:** What is the difference between Cell State ( $C_t$ ) and Hidden State ( $H_t$ ) in an LSTM?

- A. Cell State is long-term internal memory; Hidden State is the filtered output ( $H_t = O_t * \tanh(C_t)$ ) passed to predictions and next step.
- B. Hidden State is used only during training; Cell State is used only during testing.
- C. Cell State is permanent; Hidden State is deleted every step.
- D. They are identical.

**Answer: A. Cell State is long-term internal memory; Hidden State is the filtered output ( $H_t = O_t * \tanh(C_t)$ ) passed to predictions and next step.** —  $C_t$  stays internal as long-term memory;  $H_t$  is filtered through output gate  $O_t$  for immediate predictions.

**Question 85:** In an LSTM, what does the Input Gate ( $I_t$ ) combined with Candidate Cell State ( $C_{\text{tilde}_t}$ ) control?

- A. What new candidate information to write into the cell state  $C_t$ .
- B. Which parameters to freeze.
- C. The sequence batch size.
- D. The dropout rate.

**Answer: A. What new candidate information to write into the cell state  $C_t$ .** —  $I_t$  decides which features of candidate vector  $C_{\text{tilde}_t} = \tanh(\dots)$  are added to cell memory  $C_t$ .

**Question 86:** What activation function is used for all three gates (Forget  $F_t$ , Input  $I_t$ , Output  $O_t$ ) in an LSTM, and why?

- A. ReLU, because it is linear.
- B. Sigmoid (sigma), because its output range (0, 1) acts as a soft multiplicative switch or filter.

C. Softmax, because it sums to 1 across sequence steps.

D. Linear, because it does not saturate.

**Answer: B. Sigmoid (sigma), because its output range (0, 1) acts as a soft multiplicative switch or filter.** — Sigmoid bounds gate outputs between 0 (closed gate) and 1 (open gate).

**Question 87:** How many total weight matrices (excluding biases) are inside a single LSTM layer with input dim  $d$  and hidden dim  $h$ ?

A. 1

B. 4 sets of  $(W_x, W_h)$  weights = 8 weight matrices total (for F, I,  $\tilde{C}$ , O gates).

C. 16

D. 32

**Answer: B. 4 sets of  $(W_x, W_h)$  weights = 8 weight matrices total (for F, I,  $\tilde{C}$ , O gates).** — LSTM contains 4 parallel linear transformations (Forget, Input, Candidate, Output), requiring  $4 \times$  parameters of vanilla RNN.

**Question 88:** How does a Gated Recurrent Unit (GRU) simplify the LSTM architecture?

A. By eliminating all hidden states.

B. By merging Cell State and Hidden State into a single state  $H_t$ , using Reset and Update gates (2 gates total).

C. By removing activation functions.

D. By replacing recurrent layers with CNNs.

**Answer: B. By merging Cell State and Hidden State into a single state  $H_t$ , using Reset and Update gates (2 gates total).** — GRU simplifies LSTM by combining cell/hidden states and using 2 gates (Reset  $r_t$ , Update  $z_t$ ) for faster execution.

**Question 89:** In PyTorch `nn.LSTM(input_size, hidden_size)`, what format is returned by the forward call `model(inputs, (h_0, c_0))`?

A. A single scalar loss.

B. Tuple of (output,  $(h_n, c_n)$ ) where output contains hidden states for all timesteps, and  $(h_n, c_n)$  are final states.

C. A 2D image matrix.

D. A boolean mask.

**Answer: B. Tuple of (output,  $(h_n, c_n)$ ) where output contains hidden states for all timesteps, and  $(h_n, c_n)$  are final states.** — PyTorch LSTM returns sequence outputs along with final step hidden state  $h_n$  and cell state  $c_n$ .

**Question 90:** Why did LSTMs dominate NLP sequence tasks from 2011 until the rise of Transformers in 2017?

A. Because LSTMs solved vanishing gradients on long sequences while maintaining effective temporal memory.

B. Because LSTMs require no GPU memory.

C. Because LSTMs do not use backpropagation.

D. Because LSTMs convert text to images.

**Answer: A. Because LSTMs solved vanishing gradients on long sequences while maintaining effective temporal memory.** — LSTMs successfully captured long-term dependencies in language translation, speech, and sentiment analysis.

## Topic 10: Attention Mechanisms & Sequence-to-Sequence (D2L Ch 11.1–11.5)

### USEFUL STUDY INFORMATION & DEFINITIONS: Attention Mechanisms: Queries, Keys, Values & Scaled Dot-Product

#### Attention Framework Formulation (Query $q$ , Keys $k_i$ , Values $v_i$ ):

1. Attention Score:  $e_i = a(q, k_i)$  (measures similarity between query and key  $i$ )
2. Attention Weights (Softmax):  $\alpha_i = \exp(e_i) / \sum_j \exp(e_j)$  ( $\alpha_i$  in  $(0, 1)$ ,  $\sum \alpha_i = 1$ )
3. Context Vector Output:  $c = \sum_i \alpha_i v_i$  (weighted sum of values)

#### Attention Variants:

- Scaled Dot-Product Attention:  $a(q, k) = (q^T k) / \sqrt{d_k}$

Scaling Factor  $1 / \sqrt{d_k}$ : Prevents large dot product magnitudes in high dimensions  $d_k$  from driving softmax into extreme saturation regions with near-zero gradients.

- Additive (Bahdanau) Attention:  $a(q, k) = w_v^T \tanh(W_q \cdot q + W_k \cdot k)$  (useful when query and key dimensions differ).
- Multi-Head Attention: Projects  $Q, K, V$  into  $h$  parallel subspace heads, computes attention per head, concatenates outputs, and projects with  $W_o$ .

**Question 91:** What is the mathematical formula for Scaled Dot-Product Attention given Query  $Q$ , Key  $K$ , and Value  $V$ ?

- A.  $\text{Attention}(Q, K, V) = \max(Q, K, V)$
- B.  $\text{Attention}(Q, K, V) = \text{softmax}( (Q \cdot K^T) / \sqrt{d_k} ) \cdot V$
- C.  $\text{Attention}(Q, K, V) = Q + K + V$
- D.  $\text{Attention}(Q, K, V) = \tanh(Q \cdot K) \cdot V$

**Answer: B.  $\text{Attention}(Q, K, V) = \text{softmax}( (Q \cdot K^T) / \sqrt{d_k} ) \cdot V$**  — Standard scaled dot-product attention multiplies similarity  $\text{softmax}(Q \cdot K^T / \sqrt{d_k})$  by value matrix  $V$ .

**Question 92:** Why do we scale the dot products by  $1 / \sqrt{d_k}$  in Scaled Dot-Product Attention?

- A. To increase computational complexity.
- B. To prevent large dot product values in high dimensions  $d_k$  from making softmax outputs extremely peaked/saturated with near-zero gradients.
- C. To convert values into integers.
- D. To force attention weights to sum to zero.

**Answer: B. To prevent large dot product values in high dimensions  $d_k$  from making softmax outputs extremely peaked/saturated with near-zero gradients.** — As key dimension  $d_k$  grows, dot products grow in variance; scaling by  $1/\sqrt{d_k}$  stabilizes softmax gradients.

**Question 93:** In the General Attention Framework, what do Query ( $q$ ), Key ( $k$ ), and Value ( $v$ ) represent intuitively?

- A. Query is the focus probe; Keys are source item identifiers; Values are content representations retrieved based on Query-Key similarity match.
- B. Query is learning rate; Keys are weights; Values are biases.
- C. Query is output label; Keys are loss; Values are accuracy.
- D. Query is batch size; Keys are channels; Values are spatial pixels.

**Answer: A. Query is the focus probe; Keys are source item identifiers; Values are content representations retrieved based on Query-Key similarity match.** — Attention matches Query against Keys to produce a soft lookup weight distribution over Values.

**Question 94:** How did the Bahdanau (Additive) Attention mechanism revolutionize Sequence-to-Sequence Machine Translation?

- A. By forcing the encoder to compress entire input sentences into a single fixed-length bottleneck vector.
- B. By allowing decoder at each timestep to dynamically attend to all encoder hidden states, generating custom context vectors for each word.
- C. By removing the decoder entirely.
- D. By replacing text with audio.

**Answer: B. By allowing decoder at each timestep to dynamically attend to all encoder hidden states, generating custom context vectors for each word.** — Bahdanau attention broke the fixed-length bottleneck, letting decoders look back at relevant source tokens per step.

**Question 95:** What is the primary advantage of Multi-Head Attention over single-head attention?

- A. It reduces total parameter count by 90%.
- B. It projects Queries, Keys, and Values into  $h$  parallel representation subspaces, allowing the model to attend to information from different positions and relationships simultaneously.
- C. It eliminates the need for matrix multiplication.
- D. It restricts attention to immediate neighboring tokens only.

**Answer: B. It projects Queries, Keys, and Values into  $h$  parallel representation subspaces, allowing the model to attend to information from different positions and relationships simultaneously.** — Multi-head attention enables simultaneous focus on diverse relational aspects (e.g., syntactic vs semantic dependencies).

**Question 96:** In Bahdanau (Additive) Attention, what formula computes similarity score  $e(q, k)$ ?

- A.  $e(q, k) = w_v^T \cdot \tanh(W_q \cdot q + W_k \cdot k)$
- B.  $e(q, k) = q^T \cdot k$
- C.  $e(q, k) = q + k$
- D.  $e(q, k) = \max(q, k)$

**Answer: A.  $e(q, k) = w_v^T \cdot \tanh(W_q \cdot q + W_k \cdot k)$**  — Additive attention uses a single-hidden-layer feedforward network with tanh activation to measure query-key score.

**Question 97:** What is Masked Attention (or Valid Length Masking)?

- A. Masking out random neurons during training.
- B. Setting attention scores for padding tokens or invalid sequence positions to -infinity before softmax, making their attention weight zero.

- C. Masking the loss function.
- D. Hiding 50% of model parameters.

**Answer: B. Setting attention scores for padding tokens or invalid sequence positions to -infinity before softmax, making their attention weight zero.** — Replacing invalid pad positions with  $-1e9$  before softmax forces  $\exp(-1e9) \rightarrow 0$ , preventing pad tokens from receiving weight.

**Question 98:** If Query Q has shape (batch\_size, num\_queries, d\_k) and Key K has shape (batch\_size, num\_keys, d\_k), what is the shape of raw attention score matrix  $Q * K^T$ ?

- A. batch\_size x num\_queries x num\_keys
- B. batch\_size x d\_k x d\_k
- C. num\_queries x d\_k
- D. batch\_size x num\_keys x d\_k

**Answer: A. batch\_size x num\_queries x num\_keys** — Batch matrix multiplication of (num\_queries, d\_k) by (d\_k, num\_keys) produces (num\_queries, num\_keys) score matrix per batch.

**Question 99:** How are attention weights  $\alpha_i$  generated from raw alignment scores  $e_i$ ?

- A. By taking raw absolute values.
- B. By applying the Softmax function across all keys:  $\alpha_i = \exp(e_i) / \sum_j \exp(e_j)$ .
- C. By thresholding at zero.
- D. By dividing by batch size.

**Answer: B. By applying the Softmax function across all keys:  $\alpha_i = \exp(e_i) / \sum_j \exp(e_j)$ .** — Softmax converts raw alignment scores into a valid probability distribution that sums to 1.0.

**Question 100:** Why is Self-Attention computationally  $O(N^2)$  with respect to sequence length N?

- A. Because every token in the sequence computes a dot product similarity score against every other token in the sequence (N x N pairwise matrix).
- B. Because it uses 2D convolutions.
- C. Because it unrolls across 100 epochs.
- D. Because of batch normalization.

**Answer: A. Because every token in the sequence computes a dot product similarity score against every other token in the sequence (N x N pairwise matrix).** — Pairwise comparisons between N queries and N keys require evaluating an N x N score matrix.

## Topic 11: Transformer Architecture & Vision Transformers (D2L Ch 11.7–11.9)

### USEFUL STUDY INFORMATION & DEFINITIONS: Transformer Architecture & Vision Transformer (ViT)

#### Transformer Architecture Mechanics ('Attention Is All You Need', 2017):

- Self-Attention: Queries Q, Keys K, Values V are all derived from the SAME input sequence X via linear projections ( $Q = X * W_q$ ,  $K = X * W_k$ ,  $V = X * W_v$ ).
- Positional Encoding: Since self-attention is permutation-invariant (order-agnostic), sinusoidal or learnable positional embeddings P are added to input embeddings:  $X_{in} = \text{Embed}(X) + P$ .

Sinusoidal formula:  $P_{\{pos, 2i\}} = \sin(pos / 10000^{(2i/d)})$ ,  $P_{\{pos, 2i+1\}} = \cos(pos / 10000^{(2i/d)})$

- Causal Masking (Decoder): Upper triangular mask of  $-\infty$  prevents autoregressive decoders from attending to future tokens  $t+1..T$ .

- Layer Normalization (LayerNorm): Normalizes across feature dimensions within each individual sample (unlike BatchNorm which normalizes across batch samples). Ideal for variable-length NLP sequences.

Pre-LN vs Post-LN: Pre-LN (LayerNorm before sublayer) improves training stability in deep Transformers.

#### **Vision Transformer (ViT):**

- Flattens 2D image into non-overlapping spatial patches (e.g., 16x16 pixels).
- Projects patches linearly into 1D token embeddings, prepends learnable [CLS] token, adds 1D positional embeddings, and feeds into standard Transformer Encoder.

**Question 101:** What was the landmark insight of the paper 'Attention Is All You Need' (Vaswani et al., 2017)?

- A. Recurrence (RNNs) is required for sequence processing.
- B. Self-attention mechanisms alone, without recurrence or convolutions, achieve state-of-the-art sequence modeling with full parallelization.
- C. CNNs should be stacked on top of LSTMs.
- D. Gradient descent does not work for text.

**Answer: B. Self-attention mechanisms alone, without recurrence or convolutions, achieve state-of-the-art sequence modeling with full parallelization.** — Vaswani et al. proved self-attention replaces recurrent connections, enabling  $O(1)$  sequential path length and parallel training.

**Question 102:** Why is Positional Encoding necessary in Transformer models?

- A. Because self-attention is permutation-invariant; without positional encodings, word order is completely lost.
- B. To normalize input feature scale.
- C. To calculate cross-entropy loss.
- D. To prevent overfitting.

**Answer: A. Because self-attention is permutation-invariant; without positional encodings, word order is completely lost.** — Self-attention treats input tokens as an unordered set; positional encodings inject sequence order context into token vectors.

**Question 103:** How does Layer Normalization (LayerNorm) differ from Batch Normalization (BatchNorm)?

- A. BatchNorm normalizes across batch samples per feature; LayerNorm normalizes across feature dimensions independently per sample.
- B. LayerNorm is used only for images.
- C. BatchNorm requires no learnable parameters.
- D. They are identical.

**Answer: A. BatchNorm normalizes across batch samples per feature; LayerNorm normalizes across feature dimensions independently per sample.** — LayerNorm operates per individual sequence sample, making it invariant to mini-batch size and ideal for variable sequence lengths.

**Question 104:** What is Causal Masking (Look-Ahead Masking) in autoregressive Transformer Decoders (e.g., GPT)?

- A. Masking out 15% of random tokens.
- B. Setting upper-triangular self-attention scores to -infinity so token  $t$  can only attend to previous tokens  $1..t$  and not future tokens.
- C. Disabling layer normalization.
- D. Dropping 50% of decoder layers.

**Answer: B. Setting upper-triangular self-attention scores to -infinity so token  $t$  can only attend to previous tokens  $1..t$  and not future tokens.** — Causal masking prevents information leak from future tokens  $t+1..T$  during autoregressive language generation.

**Question 105:** In a Vision Transformer (ViT), how is a 224x224 RGB image converted into sequence tokens for the Transformer Encoder?

- A. By passing the image through 50 ResNet conv layers first.
- B. By dividing the image into non-overlapping 16x16 pixel patches, flattening each patch into a 1D vector, and linearly projecting to embedding dim.
- C. By computing color histograms per row.
- D. By taking the 2D Fourier Transform.

**Answer: B. By dividing the image into non-overlapping 16x16 pixel patches, flattening each patch into a 1D vector, and linearly projecting to embedding dim.** — ViT treats 16x16 image patches as 'visual words' (tokens), converting 224x224 images into a sequence of 196 patch tokens.

**Question 106:** What role does the prepended [CLS] token serve in Vision Transformers (ViT) and BERT?

- A. It marks sentence end.
- B. It acts as an aggregate global sequence representation, interacting with all patches/tokens via self-attention to drive final classification.
- C. It stores gradient norms.
- D. It replaces missing pixels.

**Answer: B. It acts as an aggregate global sequence representation, interacting with all patches/tokens via self-attention to drive final classification.** — The [CLS] token collects sequence-wide contextual information across encoder layers to feed classification heads.

**Question 107:** What is the key advantage of Pre-LN (LayerNorm applied BEFORE sublayers) over Post-LN in deep Transformers?

- A. Pre-LN provides an uninhibited residual stream gradient highway, dramatically improving training stability in deep 100+ layer Transformers.
- B. Pre-LN doubles inference speed.
- C. Pre-LN eliminates self-attention.
- D. Pre-LN converts decoders into encoders.

**Answer: A. Pre-LN provides an uninhibited residual stream gradient highway, dramatically improving training stability in deep 100+ layer Transformers.** — Normalizing inputs before attention/FFN sublayers maintains clean residual identity paths for gradient flow.

**Question 108:** What are the two sublayers contained inside every standard Transformer Encoder Block?

- A. Conv2D layer and Max Pooling layer.
- B. Multi-Head Self-Attention sublayer and Position-wise Feed-Forward Network (FFN) sublayer, each surrounded by Residual Connections & LayerNorm.
- C. LSTM cell and GRU cell.
- D. Linear regression and Softmax.

**Answer: B. Multi-Head Self-Attention sublayer and Position-wise Feed-Forward Network (FFN) sublayer, each surrounded by Residual Connections & LayerNorm.** — Standard encoder blocks stack Multi-Head Self-Attention followed by a 2-layer MLP (FFN) with residual additions.

**Question 109:** Why can Transformers be trained significantly faster on GPUs than RNNs for long sequence tasks?

- A. Because self-attention processes all N tokens simultaneously in parallel during forward/backward passes, whereas RNNs require N sequential steps.
- B. Because Transformers use smaller datasets.
- C. Because Transformers do not compute loss.
- D. Because Transformers run only on CPUs.

**Answer: A. Because self-attention processes all N tokens simultaneously in parallel during forward/backward passes, whereas RNNs require N sequential steps.** — Eliminating recurrent time-step dependencies allows full matrix parallelization across sequence length on GPUs.

**Question 110:** What is the maximum path length between any two token positions in a Transformer vs a Vanilla RNN of sequence length N?

- A. Transformer:  $O(1)$ ; RNN:  $O(N)$
- B. Transformer:  $O(N)$ ; RNN:  $O(1)$
- C. Transformer:  $O(N^2)$ ; RNN:  $O(N^2)$
- D. Transformer:  $O(\log N)$ ; RNN:  $O(1)$

**Answer: A. Transformer:  $O(1)$ ; RNN:  $O(N)$**  — Direct pairwise self-attention connects any two tokens in 1 step ( $O(1)$ ), whereas RNNs require traversing N sequential hidden states ( $O(N)$ ).

## Topic 12: Natural Language Pretraining & Multimodal Models: BERT & CLIP (D2L Ch 15.8, CLIP)

### USEFUL STUDY INFORMATION & DEFINITIONS: NLP Pretraining (BERT) & Multimodal Contrastive Models (CLIP)

#### BERT Architecture & Pretraining Objectives (Devlin et al., 2018):

- Architecture: Bidirectional Transformer Encoder-only model.

- Objective 1: Masked Language Modeling (MLM): Randomly masks 15% of input tokens with [MASK] and predicts original vocabulary token using bidirectional context.
- Objective 2: Next Sentence Prediction (NSP): Predicts whether Sentence B logically follows Sentence A (binary classification using [CLS] token).
- Input Representation: Token Embeddings + Segment Embeddings (Sentence A vs B) + Positional Embeddings.

**CLIP Multimodal Pretraining (Radford et al., OpenAI 2021):**

- Architecture: Dual Encoders (Vision Encoder e.g., ViT/ResNet + Text Encoder e.g., Transformer).
- Contrastive Learning Objective: Maximizes cosine similarity of matching (image, text) pairs along diagonal of  $N \times N$  batch matrix while minimizing off-diagonal non-matching pairs.
- Zero-Shot Classification: Prompts text encoder with 'a photo of a {class}' to generate target class embeddings, classifying images by highest cosine similarity.

**Question 111:** What type of Transformer architecture is BERT (Bidirectional Encoder Representations from Transformers)?

- A. Decoder-only architecture (like GPT)
- B. Bidirectional Encoder-only architecture
- C. Full Encoder-Decoder sequence-to-sequence model
- D. Convolutional network without attention

**Answer: B. Bidirectional Encoder-only architecture** — BERT uses unmasked Transformer Encoders to build bidirectional contextual representations.

**Question 112:** How does the Masked Language Modeling (MLM) pretraining task in BERT enable deep bidirectional context?

- A. By predicting the next word left-to-right only.
- B. By randomly masking 15% of input tokens and training the network to predict masked tokens using unmasked context from BOTH left and right directions.
- C. By translating English to French.
- D. By deleting entire sentences.

**Answer: B. By randomly masking 15% of input tokens and training the network to predict masked tokens using unmasked context from BOTH left and right directions.** — MLM allows full bidirectional self-attention across unmasked tokens without target token leakage.

**Question 113:** What three embedding vectors are summed together to create the final input representation for BERT?

- A. Token Embeddings + Segment Embeddings + Positional Embeddings
- B. RGB Embeddings + HSV Embeddings + Depth Embeddings
- C. Word2Vec + GloVe + FastText
- D. Convolutional + Pooling + Linear

**Answer: A. Token Embeddings + Segment Embeddings + Positional Embeddings** — BERT input representation combines token identity, segment ID (sentence A vs B), and 1D positional position.

**Question 114:** How is BERT fine-tuned for a downstream single-text classification task (e.g., Sentiment Analysis)?

- A. Re-pretraining the entire model from scratch on 1 billion words.
- B. Attaching a lightweight classification head (Dense layer) to the final hidden state of the [CLS] token and training end-to-end on target labels.
- C. Freezing all weights and deleting the encoder.
- D. Converting text into images.

**Answer: B. Attaching a lightweight classification head (Dense layer) to the final hidden state of the [CLS] token and training end-to-end on target labels.** — The aggregate [CLS] contextual embedding feeds a task-specific output head, updating pre-trained weights end-to-end.

**Question 115:** How does CLIP (Contrastive Language-Image Pre-training) train its image and text encoders?

- A. Using manual bounding box object labels.
- B. Using a contrastive loss on 400M internet (image, text) pairs to maximize cosine similarity for matching pairs while minimizing non-matching pairs in a shared embedding space.
- C. Using mean squared error on pixel values.
- D. Using decision trees.

**Answer: B. Using a contrastive loss on 400M internet (image, text) pairs to maximize cosine similarity for matching pairs while minimizing non-matching pairs in a shared embedding space.** — CLIP uses symmetric contrastive loss over  $N \times N$  image-text similarity matrices to construct a joint multimodal vector space.

**Question 116:** How does CLIP perform Zero-Shot Image Classification on a new target dataset without task-specific training?

- A. By retraining the vision encoder.
- B. By passing text prompt labels 'a photo of a {category}' into the text encoder to build class embeddings, then predicting the class with highest cosine similarity to the image embedding.
- C. By running k-means clustering on pixels.
- D. By converting images to ASCII text.

**Answer: B. By passing text prompt labels 'a photo of a {category}' into the text encoder to build class embeddings, then predicting the class with highest cosine similarity to the image embedding.** — CLIP matches target image vectors against text embeddings of candidate category prompts in its shared joint space.

**Question 117:** What is Prompt Engineering in the context of zero-shot CLIP classification?

- A. Writing Python code to train GPUs.
- B. Designing and optimizing descriptive text prompt templates (e.g., 'a photo of a {class}, a type of pet') to improve zero-shot text-image embedding alignment.
- C. Setting the learning rate schedule.
- D. Compressing model weights.

**Answer: B. Designing and optimizing descriptive text prompt templates (e.g., 'a photo of a {class}, a type of pet') to improve zero-shot text-image embedding alignment.** — Contextual

prompt templates eliminate ambiguity, improving text encoder alignment with pre-trained visual concepts.

**Question 118:** What is a major limitation of traditional static word embeddings (e.g., Word2Vec or GloVe) compared to contextual models like BERT?

- A. Word2Vec requires 100x more GPU memory.
- B. Static embeddings assign a single fixed vector to a word regardless of context, failing to handle polysemy (e.g., 'bank' river vs 'bank' financial).
- C. Word2Vec cannot be computed.
- D. Static embeddings only work for French.

**Answer: B. Static embeddings assign a single fixed vector to a word regardless of context, failing to handle polysemy (e.g., 'bank' river vs 'bank' financial).** — BERT dynamically computes contextual embeddings based on surrounding sentence words, resolving polysemous meanings.

**Question 119:** What was the purpose of the Next Sentence Prediction (NSP) task during BERT pretraining?

- A. To train the model to understand inter-sentence logical relationships by predicting if Sentence B follows Sentence A.
- B. To generate long stories autoregressively.
- C. To count sentence punctuation marks.
- D. To compress model size.

**Answer: A. To train the model to understand inter-sentence logical relationships by predicting if Sentence B follows Sentence A.** — NSP explicitly trains the [CLS] representation to capture sentence-pair logical relationships for tasks like NLI and QA.

**Question 120:** What key impact do multimodal models like CLIP have on computer vision applications?

- A. They bridge vision and language into a unified embedding space, enabling flexible zero-shot classification and text-to-image search without task-specific labeled data.
- B. They eliminate the need for GPUs.
- C. They replace all loss functions with linear regression.
- D. They restrict vision models to grayscale images.

**Answer: A. They bridge vision and language into a unified embedding space, enabling flexible zero-shot classification and text-to-image search without task-specific labeled data.** — Unified multimodal representations enable zero-shot transfer, open-vocabulary classification, and cross-modal retrieval.